MODE

Multi-Organization Dynamics Engine

Exploratory White Paper v1

Author: Kimberly Orihuela
Project Classification: Experimental Computational Interaction Framework
Status: Exploratory / Prototype Research
Repository: https://github.com/kimopharr-lgtm/MODE---Multi-Halo-Orbit-Dynamics-Engine

---

Abstract

MODE (Multi-Organization Dynamics Engine) is an experimental computational interaction framework exploring how deterministic balancing rules between many interacting particles can generate persistent emergent organization behaviors.

The project investigates whether combinations of attraction-like, repulsion-like, velocity-dependent, and dispersive interaction terms can produce stable or semi-stable organizational regimes including:

- bounded core-halo structures
- orbit-like motion
- compression and rebound behavior
- collision reorganization
- layered swarm organization
- path curvature and lensing-style trajectories
- adaptive many-body dynamics

MODE is not presented as a replacement for Newtonian physics, General Relativity, or established physical theory.

Instead, MODE is positioned as an exploratory procedural interaction architecture intended for experimentation across:

- procedural simulation
- VFX and gaming
- swarm dynamics

- emergent systems research
- synthetic organizational behavior
- computational art and visualization
- adaptive interaction modeling

The framework currently exists as a deterministic discrete-time simulation environment implemented primarily in Python.

---

1. Introduction

Many procedural systems rely heavily on:

- scripted animation
- predefined trajectories
- grid-constrained simulation
- force simplifications
- rigid behavioral templates

MODE explores an alternative direction:

> Can persistent large-scale organization emerge from relatively simple interaction balancing rules operating across many interacting particles?

The project began as an exploratory many-body systems experiment investigating whether balanced interaction stacks could generate visually and structurally interesting organizational regimes without relying on fixed scripted paths.

Over time, multiple repeatable behavioral regimes emerged including:

- stable inward aggregation
- bounded organization
- orbit-like motion
- collision restructuring
- layered halo persistence
- dynamic compression/rebound cycles

The project evolved into a broader investigation of procedural emergent motion and interaction-driven organization.

---

2. Project Goals

MODE currently explores five primary research directions.

2.1 Procedural Emergent Motion

Investigating whether interaction-rule-driven systems can generate visually compelling motion without frame-by-frame animation.

2.2 Persistent Organizational Regimes

Exploring conditions under which many-body systems avoid:

- total collapse
- total dispersion

and instead maintain bounded or semi-stable structures.

2.3 Adaptive Interaction Architectures

Studying how layered interaction terms influence:

- swarm coherence
- collision response
- clustering
- orbit persistence
- structural reorganization

2.4 Visualization and Simulation

Evaluating the framework as a possible procedural simulation or VFX/gaming tool.

2.5 Exploratory Systems Research

Using the framework as a sandbox for investigating emergent organizational behaviors in deterministic many-body systems.

---

3. Current Framework Architecture

MODE is implemented as a deterministic discrete-time many-body simulation.

Each particle possesses:

- position
- velocity
- phase/class properties
- interaction relationships with nearby particles

The framework updates particle states iteratively over time using balancing interaction terms.

3.1 Canonical Interaction Structure

The core interaction decomposition currently used in MODE is:

$F_{ij} = (A_{ij} - R_{ij} - D_{ij} - L_{ij}) \hat{r}_{ij}$

Where:

- A_{ij} = attraction-like interaction
- R_{ij} = short-range repulsive stabilization
- D_{ij} = velocity-dependent repulsion
- L_{ij} = long-range dispersive pressure
- $\hat{r}_{ij}$ = normalized directional vector

This interaction stack attempts to balance:

- coherence
- stabilization
- separation
- dispersive organization

within the same system.

---

4. Primary Interaction Layers

4.1 Attraction Layer

The attraction layer encourages:

- clustering
- coherence
- organization
- orbit support

Example canonical form:

$A_{ij} = (K_{LL} \Phi_{ij}) / (d_{ij}^2 + S)$

Where:

- K_{LL} = interaction strength
- Φ_{ij} = compatibility/phase factor
- S = softening parameter

4.2 Short-Range Stabilization

The repulsive stabilization layer prevents immediate collapse at short distances.

Example canonical form:

$R_{ij} = B / (d_{ij}^4 + \varepsilon)$

This layer helps support:

- finite structure formation
- collision survival
- anti-collapse behavior

4.3 Velocity-Dependent Repulsion

MODE includes a dynamic stabilization term dependent on relative velocity.

Example canonical form:

$D_{ij} = M(1 - \exp(-||v_j - v_i||^2 / V_0^2))$

This term contributes to:

- high-energy stabilization

- collision rebound
- dynamic pressure balancing
- adaptive interaction behavior

This is currently one of the framework's most distinctive architectural features.

4.4 Long-Range Dispersive Pressure

MODE also includes a long-range dispersive term.

Example canonical form:

$L_{ij} = \Lambda \exp(-(d_{ij} / R)^2)$

This term helps support:

- halo persistence
- anti-overcompression
- layered organization
- broader structural spacing

---

5. Dynamical Evolution

The simulation evolves iteratively using discrete-time updates.

Velocity Update

$v_i(t+1) = \gamma v_i(t) + F_i \Delta t$

Position Update

$x_i(t+1) = x_i(t) + v_i(t+1) \Delta t$

Where:

- γ = damping coefficient
- Δt = timestep

This creates a deterministic iterative dynamical system.

6. Current Observed Behavioral Regimes

MODE has produced multiple repeatable organizational regimes under different parameter balances.

6.1 Collapse Regimes

Attraction-dominated systems frequently collapse inward into dense core structures.

6.2 Dispersive Regimes

Repulsion-dominated systems disperse outward and lose organization.

6.3 Balanced Core-Halo Organization

Under tuned balancing conditions, systems may maintain:

- persistent core regions
- extended halo structures
- bounded organization
- limited escape behavior

6.4 Orbit-Like Motion

Certain parameter windows generate persistent circulating motion around central organizing regions.

6.5 Compression/Rebound Behavior

Some regimes demonstrate:

- inward compression
- dynamic rebound
- breathing-style structures

6.6 Collision Reorganization

Colliding structures may:

- reorganize
- survive impact
- reform layered organization

rather than immediately dispersing.

6.7 Path Curvature and Lensing-Style Motion

Some trajectories exhibit:

- curved paths
- directional bending
- lensing-like motion behavior

within the procedural interaction environment.

---

7. Current Validation State

MODE currently exists as an exploratory computational framework.

The project has demonstrated:

- repeatable simulation outputs
- deterministic seeded runs
- named benchmark branches
- reproducible behavioral categories
- bounded organization regimes
- stable orbit-like windows

However, MODE has not yet demonstrated:

- validated physical equivalence to real-world gravity
- predictive astrophysical capability
- experimentally verified physical laws
- peer-reviewed scientific validation
- production-grade computational optimization

The framework should therefore currently be interpreted as:

an exploratory emergent interaction architecture

rather than a validated physical theory.

---

8. Reproducibility and Benchmarking

MODE already includes multiple reproducibility-oriented components.

Existing Reproducibility Features

- fixed seeds
- named test branches
- canonical parameter sets
- deterministic updates
- PASS/FAIL classifications
- benchmark outputs
- saved GIF demonstrations

Current Metrics Tracked

Examples include:

- average radius
- core count
- halo count
- escaped particles
- virial-style ratios
- kinetic behavior

---

9. Potential Application Areas

MODE is currently exploratory and not production deployed.

However, possible future application areas may include:

9.1 VFX and Gaming

- procedural particle systems

- adaptive swarm motion
- cinematic anomaly effects
- synthetic environmental behavior
- orbiting visual structures
- interaction-driven animation

9.2 Computational Art

- emergent visualization
- generative simulation art
- procedural animation systems

9.3 Educational Simulation

- many-body interaction visualization
- emergent systems demonstrations
- procedural organization exploration

9.4 Swarm and Coordination Research

- organizational dynamics
- layered interaction behavior
- adaptive many-agent systems

9.5 Experimental Computational Research

- synthetic interaction frameworks
- nonlinear dynamical exploration
- emergent structure analysis

These possibilities remain exploratory.

---

10. Current Limitations

MODE currently has several important limitations.

10.1 No Physical Validation

The framework is not experimentally validated as a physical replacement theory.

10.2 Parameter Sensitivity

Behavior depends heavily on parameter balancing.

Different parameter regions may produce:

- collapse
- dispersion
- bounded organization
- unstable oscillation

10.3 Computational Scaling

Large many-body simulations may become computationally expensive.

Further optimization work is still required.

10.4 No Formal Stability Proofs

The framework currently lacks:

- rigorous stability proofs
- conserved quantity analysis
- formal equilibrium derivations

10.5 Limited Statistical Validation

Current testing has focused primarily on exploratory deterministic runs rather than large statistical sampling studies.

---

11. Future Research Directions

Potential future research directions include:

11.1 Stability Atlases

Mapping behavioral regimes across parameter spaces.

11.2 Scaling Analysis

Studying:

- particle count scaling
- halo persistence scaling
- collapse thresholds
- escape behavior

11.3 Conserved Quantity Analysis

Investigating:

- momentum drift
- angular momentum
- energy behavior
- dissipative dynamics

11.4 GPU Acceleration

Exploring:

- CUDA implementations
- parallel processing
- high-particle-count optimization

11.5 Procedural VFX Integration

Testing possible integration into:

- Unreal Engine
- Unity
- Houdini
- procedural simulation pipelines

---

12. Current Classification

The safest current classification for MODE is:

> formalized exploratory computational many-body interaction framework with repeatable emergent organization regimes

The project overlaps conceptually with:

- nonlinear dynamics
- swarm systems
- active matter analogs
- procedural simulation
- emergent systems
- computational visualization
- synthetic organization research

---

13. Conclusion

MODE is an exploratory procedural interaction framework investigating how deterministic balancing rules can generate persistent emergent organization behaviors in many-body systems.

The project has demonstrated:

- repeatable procedural organization
- orbit-like regimes
- bounded core-halo structures
- collision reorganization
- compression/rebound dynamics
- visually compelling emergent motion

The framework remains experimental and exploratory.

No claims are made that MODE replaces established physical theories or has achieved validated predictive physics.

However, the project demonstrates that relatively simple layered interaction architectures may generate rich organizational behaviors with potential relevance to:

- procedural simulation
- VFX and gaming
- emergent systems research
- computational visualization

- adaptive swarm experimentation

Further work is required in:

- benchmarking
- optimization
- formal analysis
- scaling studies
- reproducibility testing
- production integration

MODE currently serves as:

> a structured exploratory sandbox for studying deterministic emergent organization behavior.

Appendix A — Current Canonical Behavioral Principle

One of the central observed principles within MODE is:

attraction only → collapse

repulsion only → dispersion

balanced interaction stack → persistent organization

This principle remains empirical and exploratory.

Appendix B — Repository

GitHub Repository:

https://github.com/kimopharr-lgtm/MODE---Multi-Halo-Orbit-Dynamics-Engine

---

Appendix C — Current Project Status

Current maturity level:

- exploratory computational framework
- reproducible prototype environment
- procedural simulation research
- early-stage architecture formalization

Not yet:

- validated scientific theory
- production simulation platform
- commercial engine
- peer-reviewed physical model

Appendix D — Benchmarks

Current benchmark categories explored within MODE include:

Organizational Regimes
- bounded organization
- core-halo persistence
- orbit-like persistence
- collapse regimes
- dispersive regimes
- compression/rebound dynamics
- collision restructuring

Deterministic Features
- seeded repeatability
- parameter-sensitive transitions
- deterministic iterative evolution
- reproducible behavioral categories

Metrics Tracked
- average radius
- core count
- halo count
- escaped particles

- virial-style ratios
- kinetic behavior

Current Exploratory Areas
- many-body organization
- nonlinear interaction dynamics
- procedural emergent motion
- swarm organization behavior
- layered interaction balancing
- deterministic procedural systems

MODE currently remains an exploratory computational framework and is not presented as validated physical theory.

MODE CANONICAL ENGINE

```
# ======================================================================
# MODE_REAL_ENGINE_OrbitGIF.py
# REAL-ENGINE ORBIT GIF
# Uses the same v1B engine force stack:
# central attractor, phase attraction, softening, repulsion,
# velocity repulsion, long-range dispersion, damping, virial thermostat
# Do NOT upload this script publicly. Share GIF only.
# ======================================================================

import os
import math
import random
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, PillowWriter

OUT_DIR = r"C:\Users\tukib\DMHOF_GIFS"
OUT_FILE = os.path.join(OUT_DIR, "REAL_ENGINE_ORBIT_LIKE_MOTION.gif")
os.makedirs(OUT_DIR, exist_ok=True)

N = 180
FRAMES = 180
STEPS_PER_FRAME = 6
SEED = 120
```

```
DT = 0.004
SOFT = 0.70

K_CENTER = 0.042
CENTER_MASS = 4.0

K_LL = 0.010

BASE_REPULSE = 0.0016
MAX_REPULSE = 0.0020

LONG_REPEL = 0.00015
LONG_RANGE = 4.0

V0 = 0.50
DAMPING = 0.9965

TARGET_VIRIAL = 0.075
THERMOSTAT_INTERVAL = 100
MAX_SCALE_UP = 1.10
MAX_SCALE_DOWN = 0.85

MAX_FORCE = 0.025

CENTER_ORBIT_RADIUS = 1.25
CENTER_ORBIT_RATE = 0.012

random.seed(SEED)
np.random.seed(SEED)

theta = np.random.uniform(0, 2 * math.pi, N)
rad = 2.5 * np.sqrt(np.random.uniform(0, 1, N))

x = rad * np.cos(theta)
y = rad * np.sin(theta)

vx = -np.sin(theta) * 0.11 + np.random.normal(0, 0.01, N)
vy = np.cos(theta) * 0.11 + np.random.normal(0, 0.01, N)

phase_values = [0, math.pi / 2, math.pi, 3 * math.pi / 2]
phase = np.array([phase_values[i % 4] for i in range(N)])
colors = np.array([i % 4 for i in range(N)])
```

```
step_counter = 0

def kinetic_energy():
    return 0.5 * np.sum(vx * vx + vy * vy)

def counts():
    r = np.sqrt(x * x + y * y)
    core = int(np.sum(r < 1.5))
    halo = int(np.sum((r >= 1.5) & (r < 6.0)))
    escaped = int(np.sum(r >= 6.0))
    avg_radius = float(np.mean(r))
    return core, halo, escaped, avg_radius

def center_position():
    angle = CENTER_ORBIT_RATE * step_counter
    cx = CENTER_ORBIT_RADIUS * math.cos(angle)
    cy = CENTER_ORBIT_RADIUS * math.sin(angle)
    return cx, cy

def step_engine():
    global x, y, vx, vy, step_counter

    fx = np.zeros(N)
    fy = np.zeros(N)
    potential = 0.0

    cx, cy = center_position()

    dx_center = x - cx
    dy_center = y - cy

    r2_center = dx_center * dx_center + dy_center * dy_center + SOFT
    r_center = np.sqrt(r2_center)

    center_force = (K_CENTER * CENTER_MASS) / r2_center

    fx += center_force * (-dx_center / r_center)
    fy += center_force * (-dy_center / r_center)

    potential += float(-np.sum((K_CENTER * CENTER_MASS) / r_center))

    for i in range(N):
        for j in range(i + 1, N):
```

```
        dx = x[j] - x[i]
        dy = y[j] - y[i]

        d2 = dx * dx + dy * dy + 1e-9
        d = math.sqrt(d2)

        ux = dx / d
        uy = dy / d

        softened = d2 + SOFT

        phase_term = 0.5 + 0.5 * math.cos(phase[i] - phase[j])

        attraction = (K_LL * phase_term) / softened

        repulsion = BASE_REPULSE / ((d2 * d2) + 0.05)

        rvx = vx[j] - vx[i]
        rvy = vy[j] - vy[i]
        rel_v2 = rvx * rvx + rvy * rvy

        dynamic_repulse = MAX_REPULSE * (
            1.0 - math.exp(-(rel_v2 / (V0 * V0 + 1e-9)))
        )

        long_term = LONG_REPEL * math.exp(-((d / LONG_RANGE) ** 2))

        net = attraction - repulsion - dynamic_repulse - long_term
        net = max(-MAX_FORCE, min(MAX_FORCE, net))

        fpx = net * ux
        fpy = net * uy

        fx[i] += fpx
        fy[i] += fpy
        fx[j] -= fpx
        fy[j] -= fpy

        potential += -attraction / max(d, 1e-9)
        potential += repulsion
        potential += dynamic_repulse
        potential += long_term

    vx = DAMPING * vx + fx * DT
```

```
    vy = DAMPING * vy + fy * DT

    x += vx * DT
    y += vy * DT

    if step_counter > 0 and step_counter % THERMOSTAT_INTERVAL == 0:
        K = kinetic_energy()

        if abs(potential) > 1e-9 and K > 1e-9:
            virial = (2 * K) / abs(potential)

            if virial < TARGET_VIRIAL:
                desired_K = TARGET_VIRIAL * abs(potential) / 2
                scale = math.sqrt(desired_K / K)
                scale = min(scale, MAX_SCALE_UP)
                vx *= scale
                vy *= scale

            elif virial > 1.0:
                vx *= MAX_SCALE_DOWN
                vy *= MAX_SCALE_DOWN

    step_counter += 1
    return cx, cy

print("================================================")
print("MODE ORBIT BRANCH METRICS RUN")
print("================================================")
print("N:", N)
print("FRAMES:", FRAMES)
print("STEPS_PER_FRAME:", STEPS_PER_FRAME)
print("SEED:", SEED)
print("K_CENTER:", K_CENTER)
print("DAMPING:", DAMPING)
print("================================================")

total_steps = FRAMES * STEPS_PER_FRAME

for s in range(total_steps + 1):
    if s % 120 == 0:
        core, halo, escaped, avg_radius = counts()
        K = kinetic_energy()
        print(
            "step:", s,
```

```
            "avg_radius:", round(avg_radius, 6),
            "core:", core,
            "halo:", halo,
            "escaped:", escaped,
            "kinetic:", round(float(K), 6)
        )

    if s < total_steps:
        step_engine()

core, halo, escaped, avg_radius = counts()

print("-----------------------------------------------")
print("FINAL_RESULTS")
print("-----------------------------------------------")
print("final_avg_radius:", round(avg_radius, 6))
print("core:", core)
print("halo:", halo)
print("escaped:", escaped)

if escaped < 20 and halo >= 50 and core >= 20:
    classification = "PASS_ORBITING_CORE_HALO"
elif escaped < 20 and (core + halo) >= int(0.85 * N):
    classification = "PASS_BOUNDED_ORBIT_STRUCTURE"
elif escaped >= int(0.50 * N):
    classification = "FAIL_DISPERSION_ESCAPE"
else:
    classification = "PARTIAL_OR_FAIL"

print("AUTO_CLASSIFICATION:", classification)
```